

BookCook

Documentación Técnica

Documento	Documentación Técnica
Aplicación	BookCook
Plataforma	Android (Flutter)
Autor	Elias Pedrosa Madrid
Curso	DAM 2025/2026
Lenguaje	Dart / Flutter
Backend	Firebase (Auth + Firestore)

Indice

1. Descripción del proyecto
2. Objetivos
3. Tecnologías y dependencias
4. Arquitectura de la aplicación
5. Estructura de archivos
6. Modelos de datos
7. Servicios — descripción y justificación
8. Pantallas principales
9. Widgets personalizados y decisiones de diseño
10. Configuración de Firebase
11. Integración con Open Food Facts API
12. Persistencia de datos
13. Seguridad
14. Problemas encontrados y soluciones
15. Posibles mejoras y ampliaciones
16. Bibliografía

1. Descripción del proyecto

BookCook es una aplicación móvil desarrollada en Flutter que permite a los usuarios gestionar sus recetas de cocina e ingredientes con información nutricional detallada.

El objetivo principal es que el usuario pueda escanear productos del supermercado para importar automáticamente sus valores nutricionales y usarlos al crear recetas con cálculos automáticos de macros.

La aplicación implementa autenticación real con Firebase Auth, almacenamiento en la nube con Firestore, y acceso a la API pública Open Food Facts para obtener datos de productos mediante su código de barras EAN.

2. Objetivos

Objetivos funcionales

- Autenticación de usuarios con email y contraseña mediante Firebase Auth
- CRUD completo de ingredientes con datos nutricionales por 100g
- CRUD de recetas con cálculo automático de macros proporcional a la cantidad
- Escaner de código de barras que rellena automáticamente los campos del ingrediente
- Sesión persistente — no es necesario iniciar sesión en cada apertura
- Sincronización en la nube — los datos están disponibles desde cualquier dispositivo

Objetivos técnicos

- Separación de responsabilidades en capas (pantallas, servicios, modelos)
- Arquitectura singleton para servicios
- Widgets reutilizables para evitar duplicación de código
- Reglas de seguridad en Firestore que garantizan aislamiento de datos por usuario

3. Tecnologías y dependencias

Lenguaje y framework

La aplicación está desarrollada completamente en Dart usando el framework Flutter. Flutter permite compilar una única base de código a Android, iOS, Web y escritorio. En este proyecto el objetivo es Android.

Dependencias principales

Paquete	Versión	Uso	Documentado
firebase core	^2.24.2	Inicialización de Firebase	Si
firebase auth	^4.16.0	Autenticacion de usuarios	Si
cloud_firestore	^4.14.0	Base de datos en la nube	Si
mobile scanner	^5.1.1	Escaner de codigo de barras	Si *
http	^1.2.1	Llamadas a Open Food Facts API	Si *
image_picker	^1.0.7	Selección de foto desde camara/galeria	Si *
flutter_launcher_icons	^0.13.1	Generación del icono de la app	Si *

Los paquetes marcados con * no fueron usados en clase y se documentan en este apartado según los criterios de corrección.

Justificación de paquetes externos

mobile_scanner: Se eligió sobre barcode_scan2 porque tiene soporte activo en 2024, mejor rendimiento en Android moderno y API más limpia con callbacks reactivos. Permite controlar la cámara (stop/start) para evitar procesamiento múltiple del mismo código.

http: Paquete oficial de Dart para peticiones HTTP. Se usa exclusivamente para la llamada GET a la API de Open Food Facts. Se eligió sobre dio por su simplicidad.

image_picker: Única solución oficial de Flutter para acceder a cámara y galería de forma unificada. Simplifica el manejo de permisos y devuelve un XFile con el path del archivo seleccionado.

flutter_launcher_icons: Herramienta de generación automática de iconos de app en todos los tamaños requeridos por Android e iOS desde una única imagen fuente de 512x512px.

4. Arquitectura de la aplicación

BookCook sigue una arquitectura en tres capas claramente diferenciadas:

Capa	Responsabilidad	Ejemplos
Presentación (Screens)	Renderizar UI y capturar interacciones del usuario. No contiene lógica de negocio.	HomeScreen, RecipeAddScreen, IngredientDetailScreen
Dominio (Services)	Lógica de negocio, validaciones y orquestación de operaciones. Aislada de Firebase.	RecipeService, IngredientService, AuthService
Datos (Firestore/API)	Acceso a datos remotos. Abstraído tras servicios de bajo nivel.	FirestoreService, OpenFoodService

Patron Singleton

Todos los servicios se implementan como Singletons. Esto garantiza que hay una única instancia de cada servicio durante todo el ciclo de vida de la app, evitando estados inconsistentes y múltiples conexiones a Firebase.

```
static final AuthService _instance =
AuthService._internal();AuthService._internal();static AuthService get instance
=> _instance;
```

Por que esta arquitectura

Esta separación de responsabilidades permite modificar la fuente de datos (por ejemplo cambiar Firestore por SQLite) sin tocar las pantallas. También facilita el testing unitario de los servicios de dominio de forma independiente a Firebase.

5. Estructura de archivos

```
lib
├── main.dart
├── app
│   ├── app.dart
│   ├── routes.dart
│   └── theme.dart
├── models
│   ├── ingredient.dart
│   └── recipe.dart
├── screens
│   ├── credits_screen.dart
│   ├── splash_screen.dart
│   ├── auth
│   │   ├── login_screen.dart
│   │   └── register_screen.dart
│   ├── home
│   │   └── home_screen.dart
│   ├── ingredients
│   │   ├── ingredient_confirm_screen.dart
│   │   ├── ingredient_detail_screen.dart
│   │   ├── ingredient_edit_screen.dart
│   │   ├── ingredient_list_screen.dart
│   │   └── ingredient_scanner_screen.dart
│   └── recipes
│       ├── recipe_add_screen.dart
│       ├── recipe_detail_screen.dart
│       └── recipe_list_screen.dart
├── services
│   ├── auth_service.dart
│   ├── firestore_service.dart
│   ├── ingredient_service.dart
│   ├── open_food_service.dart
│   ├── recipe_service.dart
│   └── storage_service.dart
├── utils
│   └── validators.dart
└── widgets
    ├── buttons
    │   └── primary_button.dart
    ├── cards
    │   ├── ingredient_card.dart
    │   └── recipe_card.dart
    ├── common
    │   ├── custom_app_bar.dart
    │   ├── logo_header.dart
    │   └── smart_image.dart
    ├── inputs
    │   └── custom_text_field.dart
    └── navigation
        └── bottom_nav_bar.dart
```

6. Modelos de datos

Ingredient

Representa un ingrediente con sus valores nutricionales por 100g. Incluye campos opcionales para micronutrientes (fibra, azúcar, sodio) y para la imagen (puede ser URL de la API o path local de cámara).

Campo	Tipo	Obligatorio	Descripción
id	String	Si	UUID único generado en creación
name	String	Si	Nombre del ingrediente
brand	String?	No	Marca del producto
imagePath	String?	No	URL (API) o path local (cámara)
calories	double	Si	Calorías por 100g
protein	double	Si	Proteínas por 100g
carbs	double	Si	Carbohidratos por 100g
fats	double	Si	Grasas por 100g
fiber	double?	No	Fibra por 100g
sugar	double?	No	Azúcar por 100g
sodium	double?	No	Sodio por 100g (mg)

Recipe y RecipeIngredient

Recipe contiene una lista de RecipeIngredient. Cada RecipeIngredient relaciona un Ingredient con la cantidad usada en la receta (String porque proviene de un TextField).

Los getters nutricionales calculan el total proporcional a la cantidad:

Si el usuario no introduce cantidad, se asumen 100g como valor por defecto (el valor base de la API).

7. Servicios

AuthService

Encapsula Firebase Auth.

Métodos: register(), login(), logout().

Los errores de Firebase (FirebaseAuthException) se traducen a mensajes legibles para el usuario mediante _translateError().

Solo captura FirebaseAuthException, los errores generales se dejan pasar para no ocultar fallos reales.

FirestoreService

Servicio de bajo nivel que abstrae el acceso a Firestore.

Estructura de datos: users/{uid}/ingredients/{docId} y users/{uid}/recipes/{docId}

Cada usuario solo puede acceder a sus propios documentos gracias al UID del usuario autenticado.

IngredientService y RecipeService

Servicios de dominio.

Mantienen una lista en memoria sincronizada con Firestore

Métodos: init() carga datos al arrancar, create/update/delete sincronizan con Firestore

Las pantallas solo interactúan con estos servicios, nunca directamente con Firestore.

Open Food Service

Llama a la API REST de Open Food Facts.

Devuelve OpenFoodResult? (null si no encuentra o hay error de red).

La clase _toDouble() convierte los valores que pueden venir como String, int o double desde el JSON de la API.

8. Pantallas principales

Pantalla	Widgets principales	Descripción
SplashScreen	Scaffold, Column, CircularProgressIndicator	Comprueba sesión activa y redirige
LoginScreen	Scaffold, CustomTextField, PrimaryButton	Formulario de acceso con validación local
RegisterScreen	Scaffold, CustomTextField, PrimaryButton	Registro de nueva cuenta
HomeScreen	Scaffold, Drawer, IndexedStack, BottomNavBar	Contenedor principal con drawer y nav
RecipeListScreen	GridView.builder, RecipeCard	Cuadrícula de recetas con buscador
RecipeAddScreen	SingleChildScrollView, DraggableScrollableSheet	Formulario con selector de ingredientes modal
RecipeDetailScreen	TabBar, TabBarView, FloatingActionButton	Detalle con tabs y botón eliminar
IngredientListScreen	GridView.builder, IngredientCard	Cuadrícula de ingredientes
IngredientScannerScreen	MobileScanner, Stack	Cámara con overlay de escaneo
IngredientConfirmScreen	SingleChildScrollView, AnimatedCrossFade	Confirmación de datos del producto
CreditsScreen	SingleChildScrollView, Column	Información del autor y tecnologías

9. Widgets personalizados y decisiones de diseño

SmartImage

Widget creado para resolver el problema de que los ingredientes tienen tres tipos de imagen posibles: URL de red (Open Food Facts), path del archivo local (cámara del dispositivo) o null (placeholder). Un único widget detecta el tipo y usa `Image.network`, `Image.file` o `Image.asset` según corresponda, con fallback al placeholder si falla la carga.

CustomTextField

`TextField` envuelto en un `Container` con estilo unificado. Recibe `label`, `hint`, `prefixIcon`, `controller` y `keyboardType`. Centraliza el estilo visual de todos los campos de la app para garantizar consistencia.

PrimaryButton

`ElevatedButton` con el color primario, bordes redondeados y padding estándar.

Se usa en todas las acciones principales (guardar, registrarse, iniciar sesión).

BottomNavBar

`BottomNavigationBar` con dos items: Recipes e Ingredients.

Implementado en `HomeScreen` usando `IndexedStack` para mantener el estado de cada pestaña al cambiar de una a otra.

DraggableScrollableSheet para el selector de ingredientes

El modal de selección de ingredientes usa `DraggableScrollableSheet` para permitir al usuario expandirlo o contraerlo arrastrando.

Tiene un tamaño inicial del 80% de la pantalla, mínimo 50% y máximo 95%

AnimatedCrossFade para micronutrientes

La sección de micronutrientes usa `AnimatedCrossFade` para la animación de expandir/contraer. Se eligió sobre `Visibility` o `if/else` porque anima suavemente la transición entre los dos estados.

10. Configuración de Firebase

Servicios utilizados

- Firebase Authentication: email y contraseña
- Cloud Firestore: base de datos NoSQL en tiempo real

Estructura de Firestore

```
users/  
  {uid}/  
    ingredients/  
      {ingredientId}: { name, brand, calories, ... }  
    recipes/  
      {recipeId}: { name, ingredients: [...], steps: [...] }
```

Reglas de seguridad

```
match /users/{userId}/{document=**} {  
  allow read, write: if request.auth != null  
    && request.auth.uid == userId;  
}
```

Estas reglas garantizan que cada usuario solo puede leer y escribir sus propios documentos. Un usuario autenticado no puede acceder a los datos de otro usuario.

11. Integración con Open Food Facts API

Open Food Facts es una base de datos abierta de productos alimenticios.

La app realiza una petición GET con el código EAN del producto escaneado:

```
GET https://world.openfoodfacts.org/api/v0/product/{barcode}.json
```

La respuesta JSON contiene los campos nutricionales por 100g en el objeto nutriments.

Los campos pueden venir con sufijo _100g o sin él, y pueden ser String, int o double, por lo que el método _toDouble() hace la conversión segura.

Campo API	Campo modelo	Tipo
product_name	name	String
brands	brand	String
image_url	imageUrl	String
nutriments.energy-kcal_100g	calories	double
nutriments.proteins_100g	protein	double
nutriments.carbohydrates_100g	carbs	double
nutriments.fat_100g	fats	double
nutriments.fiber_100g	fiber	double?
nutriments.sugars_100g	sugar	double?
nutriments.sodium_100g	sodium	double?

Si el producto no está en la base de datos (status != 1) o hay un error de red, el servicio devuelve null y la app muestra la pantalla de confirmación en modo manual.

12. Persistencia de datos

La app usa exclusivamente Firestore como capa de persistencia.

No hay base de datos local adicional.

Flujo de datos

1. El usuario inicia sesión -> Firebase Auth devuelve el UID del usuario.
2. HomeScreen llama a IngredientService.init() y RecipeService.init().
3. Cada servicio llama a FirestoreService.getAll() que lee todos los documentos de la colección del usuario en Firestore.
4. Los documentos se deserializan con fromMap() y se guardan en lista local en memoria.
5. Cada operación CRUD actualiza la lista en memoria y llama a Firestore de forma asíncrona.

13. Seguridad

- Las credenciales de Firebase (google-services.json) no se incluyen en el repositorio público.
- Las reglas de Firestore impiden que un usuario acceda a datos de otro.
- El token de sesión de Firebase Auth se gestiona automáticamente y tiene caducidad.
- Las contraseñas nunca se almacenan en Firebase solo guarda el hash.
- La app no almacena credenciales localmente en ningún momento.

14. Problemas encontrados y soluciones

Problema	Causa	Solucion aplicada
Gradle incompatible con Java 25	Error: Unsupported class file major version 69	Cambiar Gradle JDK a Java 17 en Android Studio Settings
Plugin google-services no encontrado	Sintaxis Groovy usada en Kotlin DSL	Declarar en settings.gradle.kts con apply false y aplicar en app/build.gradle.kts
Spinner infinito en registro	updateDisplayName() fallaba	Envolver updateDisplayName en try/catch propio — no bloquea el registro
Imagenes no se mostraban en listado	Image.asset no soporta URLs ni paths locales	Widget SmartImage que detecta el tipo automáticamente
Drawer no visible en pantallas hijas	Scaffold.of buscaba Scaffold hijo sin Drawer	Mover AppBar a HomeScreen y usar Builder para el contexto correcto
Cálculos nutricionales incorrectos	Sumaba valores por 100g ignorando la cantidad	Fórmula: nutriente * cantidad / 100 en el modelo Recipe